

1. Práctica 1: Actualización remota (OTA) utilizando *Things-Board*

1.1. Objetivos

- Entender la técnica de actualizaciones A/B.
- Preparar una aplicación para que sea actualizable.
- Comprender el funcionamiento de los bits de opción de arranque del procesador.

1.2. Fundamentos teóricos

Aunque los sistemas empotrados suelen tener funciones muy específicas y constantes a lo largo del tiempo, su complejidad a día de hoy hace que, en ocasiones, se cuele algún bug, o haga falta realizar algún cambio en los programas que los gobiernan.

Cuando desarrollamos desde nuestro ordenador, con un cable conectado, este proceso es muy sencillo: con un depurador (bien externo o integrado en la placa) podemos cambiar el código fácilmente. Sin embargo, al desplegar las soluciones en el mundo real, perdemos el acceso directo. Únicamente el propio controlador, a través de las interfaces que lo conecten al mundo exterior, es capaz de recibir información y, potencialmente, actualizarse.

Existen numerosas maneras de realizar estas actualizaciones remotas, aunque todas tienen algo en común: El código que actualiza la aplicación, no puede actualizarse a sí mismo, o de lo contrario se puede corromper mientras se actualiza. Para salvar esta barrera, existen numerosas técnicas, por ejemplo utilizando *bootloaders*, utilizando *launchers* con la única función de actualizar, haciendo que la propia app se actualice...

En nuestro caso vamos a utilizar una de las aproximaciones más robustas ante fallos, conocida como las actualizaciones A/B. En esta aproximación, se divide el espacio de memoria en dos zonas (A y B), de tal manera que la aplicación pueda residir en ambas. Cuando la app arranca desde A, en caso de encontrar una actualización la hará en B, y viceversa. Los bits de arranque del sistema se configuran, en cada caso, para arrancar la app más nueva. Añadiendo pasos de comprobación de integridad de los datos, se asegura, antes de cambiar el arranque, que la aplicación descargada no tiene fallos. Con todo esto, las probabilidades de *brickear* (o dejar inutilizable) el dispositivo se reducen enormemente. Este flujo general puede verse en Figura 1.

1.2.1. El problema de la compilación

Al compilar una aplicación para un sistema empotrado, normalmente se prepara con la idea de que siempre va a arrancar desde el mismo espacio en la memoria. De hecho, el código, en general no es de tipo PIC (position independent code), sino que ciertas direcciones de memoria están fijadas y no pueden cambiar. Al aplicar esta división A/B, esto sería un problema: la aplicación arranca desde dos direcciones de memoria diferentes. Es por ello que, en este tipo de aplicaciones, hay varias cosas a tener en cuenta:

- La aplicación debe ser consciente de cuál es su dirección de arranque (A o B).

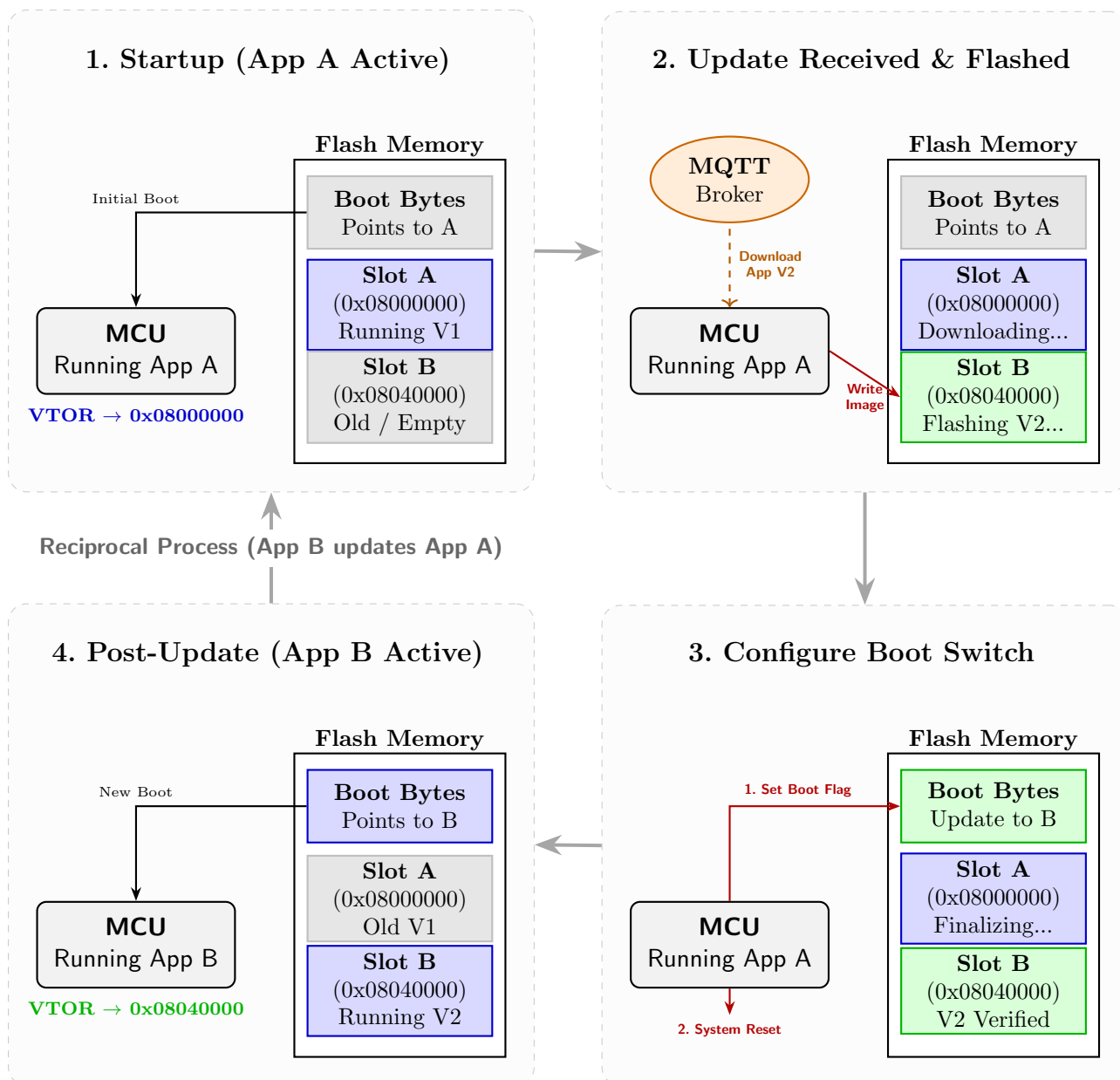


Figura 1: Diagrama de actualización A/B

- Se deben realizar dos compilaciones diferentes (A y B), cada una atacando una de las dos regiones de memoria.
- Cada compilación sólo puede residir en su zona de compilación predeterminada, con lo cual el sistema de actualizaciones deberá tenerlo en cuenta.

1.3. Desarrollo de la práctica

1.3.1. Preparación del código

El código ya está preparado para descargar la actualización OTA de *ThingsBoard* (configuraremos la interfaz web más adelante). Explicamos aquí con detalle los cambios respecto a la anterior práctica.

En la función `ESP01S_ProcessMQTT` tenemos los primeros cambios. En primer lugar, observamos que, además de comprobar en los mensajes de entrada las cadenas de actualización de `ledState`, ahora se llama a la función `OTA_ProcessJSON`. Esta función (definida en los nuevos archivos `ota.h` y `ota.c`) se encarga de parsear el aviso de nuevo firmware disponible desde el servidor. Adicionalmente, se añade una comprobación directa para mensajes recibidos con el propio firmware (a través del *topic* MQTT “v2/fw/response”).

```

1  if (MQTTDeserialize_publish(&dup, &qos, &retained, NULL, &receivedTopic,
2                                &payload, &payloadlen, packet, copy_len) == 1)
3  {
4      //Debug_Printf("[MQTT RX] %s\r\n", receivedTopic.lenstring.data);
5      // 1. Handle Binary Firmware Response Chunk
6      if (memcmp(receivedTopic.lenstring.data, "v2/fw/response", 14) == 0 &&
7          OTA_getState() == OTA_STATE_DOWNLOADING) {
8          OTA_WriteChunkAndProcess(payload, payloadlen);
9      }
10     // 2. Handle JSON Attributes (Runtime Notification or Startup Request)
11     else {
12         char json_str[128] = {0};
13         uint16_t str_len = (payloadlen < sizeof(json_str) - 1) ? payloadlen :
14                             sizeof(json_str) - 1;
15         memcpy(json_str, payload, str_len);
16
17         OTA_ProcessJSON(json_str);
18
19         // LED command processing
20         if (strstr(json_str, "\"ledState\":true"))        GPIO_PIN_WRITE(GPIOA,
21                                     7, GPIO_STATE_ONE);
22         else if (strstr(json_str, "\"ledState\":false"))    GPIO_PIN_WRITE(GPIOA,
23                                     7, GPIO_STATE_ZERO);
24     }
25 }

```

Para estar en el modo de descarga de firmware, primero este debe ser solicitado. La función `OTA_ProcessJSON` se encarga de extraer la versión y tamaño del firmware anunciado desde el servidor. La versión desde el servidor tendrá la forma 1.0.0.A o 1.0.0.B, según el *slot* que ocupe. Esto sirve para identificar dos cosas: en primer lugar, si la versión es superior a la actual (definida en `ota.h` como `#define CURRENT_FW_VERSION "1.0.0"`), y si es la adecuada para el *slot* libre. En tal caso, se procede a borrar los sectores adecuados de la **FLASH**, y a solicitar los *chunks* o trozos de actualización uno tras otro (ya que, en general, las actualizaciones son demasiado grandes como para descargarlas de golpe).

```

1 void OTA_ProcessJSON(const char *json) {
2     char *ver_ptr = strstr(json, "\"fw_version\":");
3     char *size_ptr = strstr(json, "\"fw_size\":");
4
5     if (ver_ptr && size_ptr && ota_ctx.state == OTA_STATE_IDLE) {
6         char new_version[32] = {0};
7         sscanf(ver_ptr, "\"fw_version\": \"%31[^\"]\"", new_version);
8         uint32_t new_size = (uint32_t)atoi(size_ptr + 10);
9
10        // Determine what binary suffix this device requires
11        const char *required_suffix = ((uint32_t)&_app_start ==
12            FLASH_SLOT_A_ADDR) ? "B" : "A";
13
14        // Reject the update if the incoming version tag doesn't match the
15        // inactive slot
16        if (strstr(new_version, required_suffix) == NULL) {
17            Debug_Printf("[OTA] Rejected update %s! Device is running in Slot %c
18                and requires %s binary.\r\n",
19                new_version,
20                ((uint32_t)&_app_start == FLASH_SLOT_A_ADDR) ? 'A' : 'B',
21                required_suffix);
22            return;
23        }
24
25        // Verify if version differs from running app
26        if (strcmp(new_version, CURRENT_FW_VERSION) != 0 && new_size > 0) {
27            Debug_Printf("[OTA] New Version Found: %s (Size: %lu bytes)\r\n",
28                new_version, new_size);
29
30            ota_ctx.state = OTA_STATE_DOWNLOADING;
31            ota_ctx.total_size = new_size;
32            ota_ctx.bytes_downloaded = 0;
33            ota_ctx.current_chunk = 0;
34            strncpy(ota_ctx.target_version, new_version, sizeof(ota_ctx.
35                target_version));
36
37            OTA_EraseTargetSlot();
38            OTA_RequestNextChunk();
39        }
40    }
41 }

```

Estos *chunks* se van procesando en `OTA.WriteChunkAndProcess`. Esta función va situando el puntero de escritura de la **FLASH** para ir poco a poco escribiendo el nuevo firmware, hasta completar la descarga.

```

1 void OTA_WriteChunkAndProcess(uint8_t *data, uint16_t length) {
2     OTA_WriteChunk(ota_ctx.bytes_downloaded, data, length);
3     ota_ctx.bytes_downloaded += length;
4     ota_ctx.current_chunk++;
5
6     Debug_Printf("[OTA RX] Chunk %u (%d bytes) -> Total: %lu / %lu\r\n",
7         ota_ctx.current_chunk, length, ota_ctx.bytes_downloaded, ota_ctx
8         .total_size);
9
10    if (ota_ctx.bytes_downloaded < ota_ctx.total_size) {
11        OTA_RequestNextChunk();
12    } else {
13        Debug_Printf("[OTA SUCCESS] Download Finished! Rebooting...\r\n");
14        ota_ctx.state = OTA_STATE_COMPLETE;
15        OTA_SwapBootSlotAndReset();
16    }
17 }

```

```

15     }
16 }

```

Cuando la descarga está completa, se lanza `OTA_SwapBootSlotAndReset`. Esta función tiene la importante función de calcular la nueva dirección de arranque y grabarla en los bits de opción (`rm0431 3.4.1`), para que la siguiente vez que el procesador comience, lo haga desde el nuevo *slot*. Los bits de opción de la `FLASH` permiten precisamente configurar ciertas opciones de arranque, que son tenidas en cuenta antes de que comience a ejecutar cualquier código. Así, no es la aplicación quién decide desde qué dirección se arranca, sino el propio procesador.

En nuestro procesador **STM32F723IE**, existen dos posibles direcciones de arranque (`rm0431 3.7.7`). El hecho de existir dos es para dar la opción de, al resetear, elegir cuál sirve de arranque con la pulsación (o no) de un botón. Normalmente, lo que se hace es colocar un *bootloader* en una de las direcciones iniciales, y nuestra aplicación en la otra. Esto hará que, si por algún motivo nuestra aplicación está mal escrita y no tiene comunicación con el depurador, se pueda recuperar la misma cargando el *bootloader*. En nuestra placa, al tener un depurador integrado, no tenemos que preocuparnos de ningún problema, ya que el depurador está conectado directamente por un puerto SWI (Serial Wire Interface) que puede tomar el control total del procesador aunque se quede “pillado”.

En resumen, lo que haremos será ajustar la dirección `BOOT_ADD0` en `OPTCR1` para apuntar a la aplicación que queramos arrancar. Como estos bits son delicados, no se puede escribir directamente en ellos, y hay que aplicar una secuencia de desbloqueo a través de los registros `CR` (`rm0431 3.7.5`), `KEYR` (`rm0431 3.7.2`), `OPTCR` (`rm0431 3.7.6`) y `OPTKEYR` (`rm0431 3.7.3`). Tras hacerlo, forzamos un volcado de la memoria mediante la primitiva `__DSB()`, y reseteamos el sistema desde el `AIRCR` del `SCB` (`pm0253 4.3.5`).

```

1 void OTA_SwapBootSlotAndReset(void)
2 {
3     uint32_t target_addr = OTA_GetTargetAddress();
4     // BOOT_ADD0 expects the flash address divided by 16KB (shifted right by 14
5     // bits)
6     uint16_t boot_val = (uint16_t)(target_addr >> 14);
7
8     Debug_Printf("[OTA] Swapping BOOT_ADD0 to 0x%08LU (val: 0x%04X) and
9     // resetting...\r\n",
10     // target_addr, boot_val);
11     Delay_ms(500);
12
13     Flash_Unlock(); // Uses bsp.h helper
14     Flash_Unlock_Opt();
15     Flash_WaitNotBusy(); // Uses bsp.h helper
16
17     //update target boot address
18     Flash_Update_Opt(boot_val);
19
20     Flash_WaitNotBusy();
21     Flash_Lock_Opt();
22     Flash_Lock(); // Uses bsp.h helper
23
24     // Force system reset after Data Synchronization Barrier
25     // (to flush pending mem stores)
26     __DSB();
27     SCB->AIRCR = (0x05FA0000U) | (1U << 2); // VECTKEY (0x05FA) + SYSRESETREQ (
28     // Bit 2)
29
30     while (1); // Wait for hardware reset
31 }

```

Y... listo! Tras este proceso el procesador comenzará a ejecutar en el *slot* correspondiente. Al arrancar volverá a recibir la información del *firmware* disponible y, viendo que ya está actualizado, continuará su funcionamiento de manera normal. Podemos ver un resumen gráfico de todo el proceso en la Figura 2.

1.3.2. Preparación de la compilación

Ahora viene lo interesante. Tenemos el código ya casi listo, pero faltan dos cosas: Hacer consciente a la aplicación de quién es (A o B + versión), y compilarla de dos maneras diferentes, además de generar un archivo binario que poder subir a *ThingsBoard*. Vamos a empezar por lo segundo. Para compilarla de dos formas distintas, necesitamos dos *Linker Script* diferentes. Al *Linker Script* por defecto, debemos hacerle dos variantes:

```
1  /// ARCHIVO STM32F723IEKX_FLASH_A.ld ///
```

```
2
```

```
3  /* Memories definition */
```

```
4  MEMORY
```

```
5  {
```

```
6      RAM        (xrw)      : ORIGIN = 0x20000000,    LENGTH = 256K
```

```
7      FLASH      (rx)       : ORIGIN = 0x80000000,    LENGTH = 256K
```

```
8  }
```

```
9
```

```
10 /* Export FLASH start address to C code */
```

```
11 PROVIDE(_app_start = ORIGIN(FLASH));
```

```
1  /// ARCHIVO STM32F723IEKX_FLASH_B.ld ///
```

```
2
```

```
3  /* Memories definition */
```

```
4  MEMORY
```

```
5  {
```

```
6      RAM        (xrw)      : ORIGIN = 0x20000000,    LENGTH = 256K
```

```
7      FLASH      (rx)       : ORIGIN = 0x80400000,    LENGTH = 256K
```

```
8  }
```

```
9
```

```
10 /* Export FLASH start address to C code */
```

```
11 PROVIDE(_app_start = ORIGIN(FLASH));
```

Con esto, conseguimos que la base de la **FLASH** se sitúe en dos diferentes, o bien `0x08000000` o `0x08040000`. Es importante destacar que, al hacer esta partición, contamos con solo 256KB de espacio para nuestra app (al necesitar dos huecos) en lugar de los 512KB que teníamos antes. Otras aproximaciones a las actualizaciones, como la de incluir un pequeño actualizador en lugar de ser la aplicación completa la que actualiza, dejan más espacio libre. En cualquier caso, para nuestros propósitos y tamaños de librería, es suficiente (el binario final ocupa unos 30KB).

Además de ajustar la dirección inicial, exportamos un símbolo `_app_start` que podemos usar desde el código y nos indica la dirección de comienzo con la que hemos compilado el binario, para que desde ejecución seamos capaces de identificarla. Por otro lado, en `ota.h` tenemos las definiciones para poder hacer las actualizaciones:

```
1  #define CURRENT_FW_VERSION "1.0.0"
```

```
2  #define FLASH_SLOT_A_ADDR  0x08000000U
```

```
3  #define FLASH_SLOT_B_ADDR  0x08040000U
```

Así, en diferentes puntos del código, podemos identificar versión y *slot* para tomar diferentes decisiones:

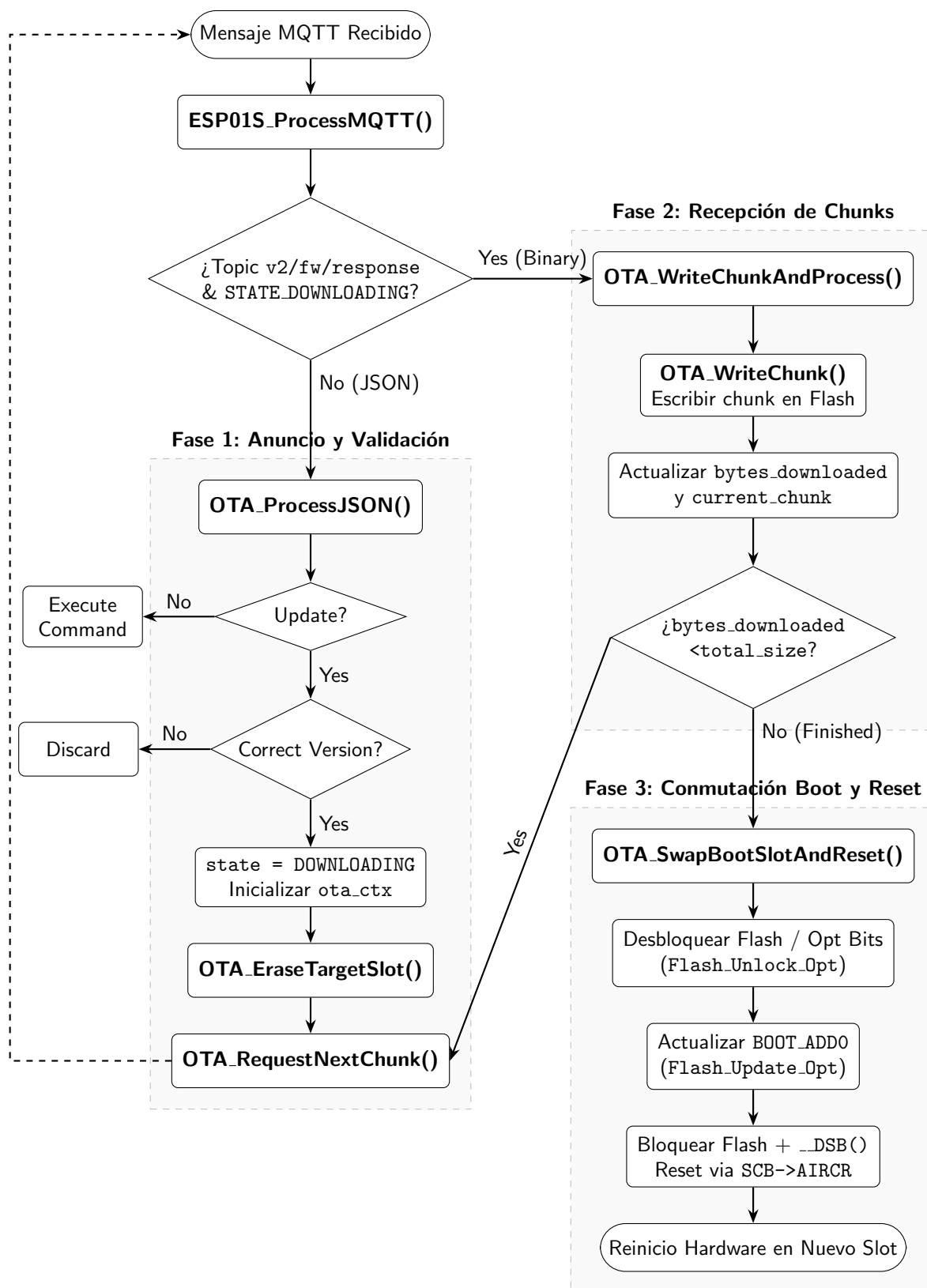


Figura 2: Diagrama de flujo de recepción OTA

```

1  // en main.c
2  Debug_Printf("[BOOT] Running App v%s from Address: 0x%08X\r\n",
3      CURRENT_FW_VERSION, (unsigned int)(uint32_t)&_app_start);
4
5  // en ota.c -> OTA_ProcessJSON
6  // Determine what binary suffix this device requires
7  const char *required_suffix = ((uint32_t)&_app_start == FLASH_SLOT_A_ADDR) ? "B"
8      : "A";
9
10 // Reject the update if the incoming version tag doesn't match the inactive slot
11 if (strstr(new_version, required_suffix) == NULL) {
12     Debug_Printf("[OTA] Rejected update %s! Device is running in Slot %c and
13         requires %s binary.\r\n",
14         new_version,
15         ((uint32_t)&_app_start == FLASH_SLOT_A_ADDR) ? 'A' : 'B',
16         required_suffix);
17     return;
18 }
19
20 // en ota.c -> OTA_EraseTargetSector
21 uint32_t target_addr = OTA_GetTargetAddress();
22
23 if (target_addr == FLASH_SLOT_B_ADDR) {
24     Debug_Printf("[OTA] Erasing Slot B (0x08040000)...\r\n");
25     Flash_EraseSector(6);
26     Flash_EraseSector(7);
27 } else {
28     Debug_Printf("[OTA] Erasing Slot A (0x08000000)...\r\n");
29     Flash_EraseSector(0);
30     // ...
31
32 // en ota.c -> OTA_SwapBootSlotAndReset
33 uint32_t target_addr = OTA_GetTargetAddress();
34 // BOOT_ADD0 expects the flash address divided by 16KB (shifted right by 14 bits)
35 uint16_t boot_val = (uint16_t)(target_addr >> 14);
36
37 Debug_Printf("[OTA] Swapping BOOT_ADD0 to 0x%08LU (val: 0x%04X) and resetting
38     ...\r\n",
39     target_addr, boot_val);

```

Para preparar estas dos compilaciones, debemos añadir los dos *linker script* al proyecto, y a continuación crear dos nuevas configuraciones de *build* dando click derecho al proyecto, **build configurations >manage >new**, tras lo que veremos un diálogo como el de la Figura 3. Hay que crear 2, una llamada “SLOT A” y otra “SLOT B”.

A continuación, en las propiedades del proyecto, entramos a **C/C++ Build >Settings >MCU/MPU GCC Linker**. Tenemos que seleccionar arriba la configuración adecuada (A o B) y cambiar el nombre del *linker script* al que acaba en *_A* o *_B* que hemos creado anteriormente. Esto se muestra en la Figura 4

También debemos activar la opción de generar binario desde la misma configuración del proyecto, en **C/C++ Build >Settings >MCU/MPU Settings** (Figura 5).

Ahora ya podemos compilar los dos binarios diferentes. Para ello basta darle al martillo de arriba, en la configuración “SLOT A” o “SLOT B”. Veremos aparecer en nuestro proyecto dos carpetas, con sendos nombres, que contendrán los archivos compilados de cada una de las versiones. Para depurar, tendremos que ir a **Debug >Debug Configurations** y elegir el binario

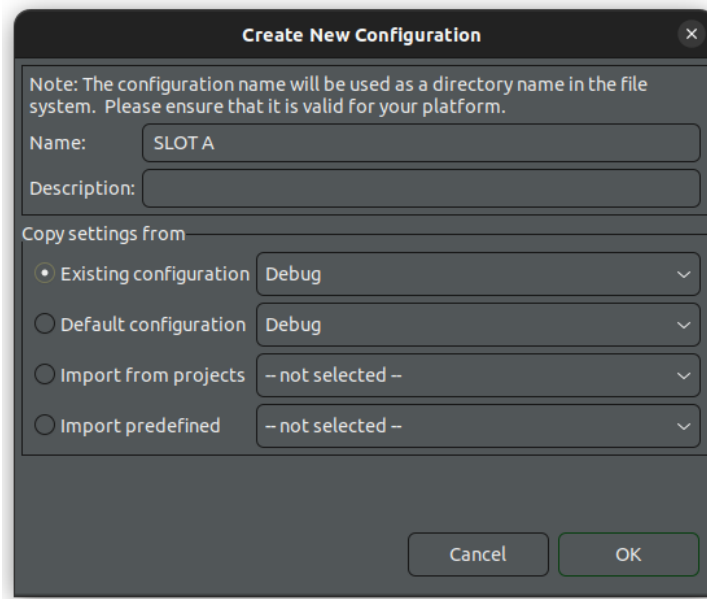


Figura 3: Creando una nueva configuración de *build*

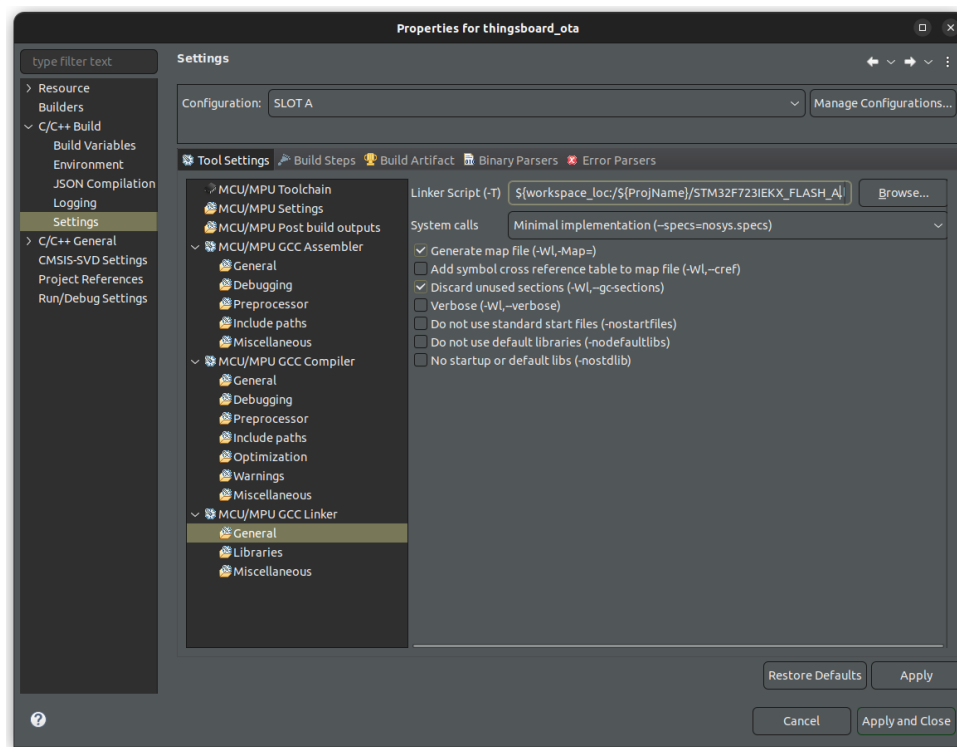


Figura 4: Configuración de linker script

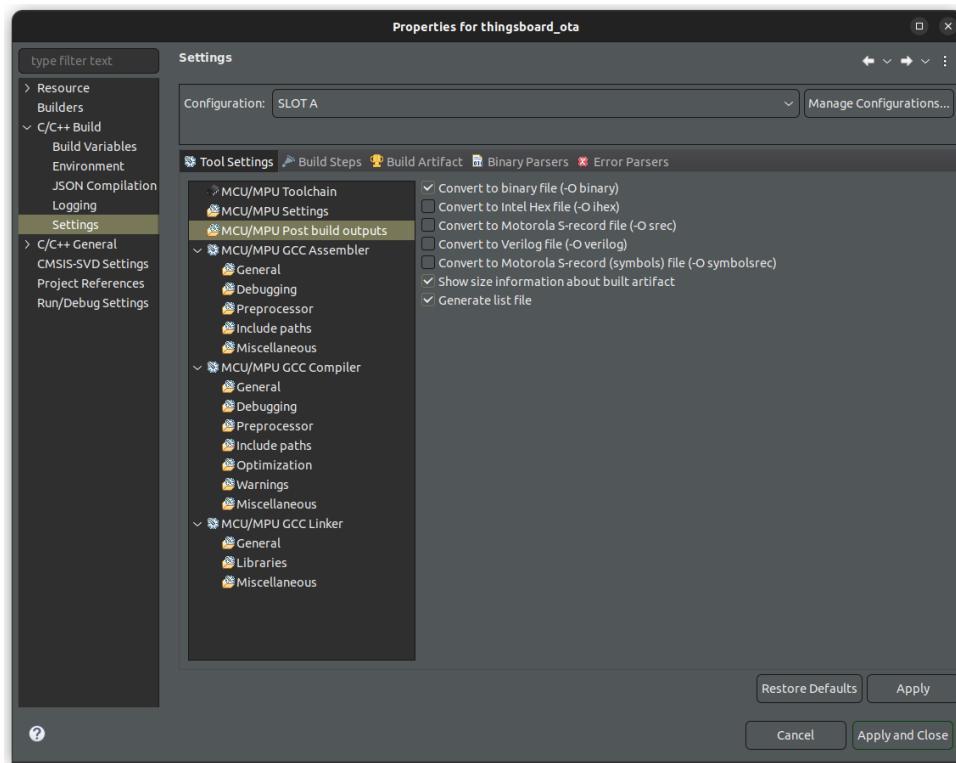


Figura 5: Configuración para generar archivo binario

que queremos depurar como muestra la Figura 6.

Al arrancar, deberíamos ver por terminal el siguiente mensaje, indicando que todo ha ido bien:

```
1 [B00T] Running App v1.0.0 from Address: 0x08000000
2 [ESP] Testing ESP01S AT Communication...
3 ...
```

1.3.3. Configuración en *Thingsboard*

Tras tener preparadas las dos compilaciones, podemos subirlas a *ThingsBoard*. Para ello, hay que ir a la sección **Advanced Features >OTA Updates** y añadir dos nuevos paquetes de Firmware. Tendremos que indicar un nombre y versión, además de asociarlo con un perfil (por defecto en nuestro caso), y subir el binario. Todo el proceso se muestra en la Figura 7

Podemos subir diferentes versiones, y seleccionar en cada momento la que nuestro dispositivo debería tener. En este caso, lo hacemos desde el apartado de **Entities >Devices**, donde podremos seleccionar, para nuestro dispositivo (ver Figura 8), el firmware que queramos y será anunciado mediante **MQTT**.

Por ejemplo, podemos seleccionar el firmware “A” (**Devices >Details >Assigned Firmware**) y arrancar la aplicación, desde STM32CubeIDE, también en el slot A. Si abrimos una terminal para ver los mensajes de depuración, deberíamos observar lo siguiente:

```
1 [B00T] Running App v1.0.0 from Address: 0x08000000
2 [ESP] Testing ESP01S AT Communication...
3 [ESP] Disabling Echo...
```

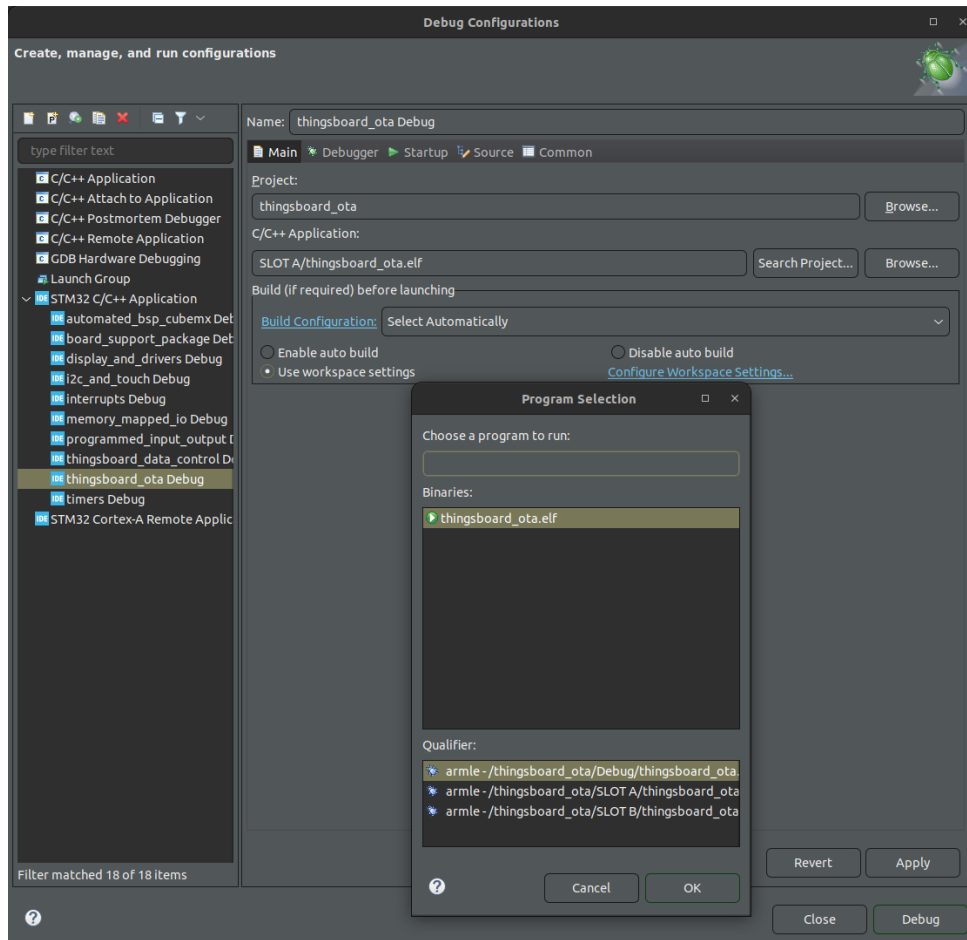


Figura 6: Configuración para cambiar el binario de depuración

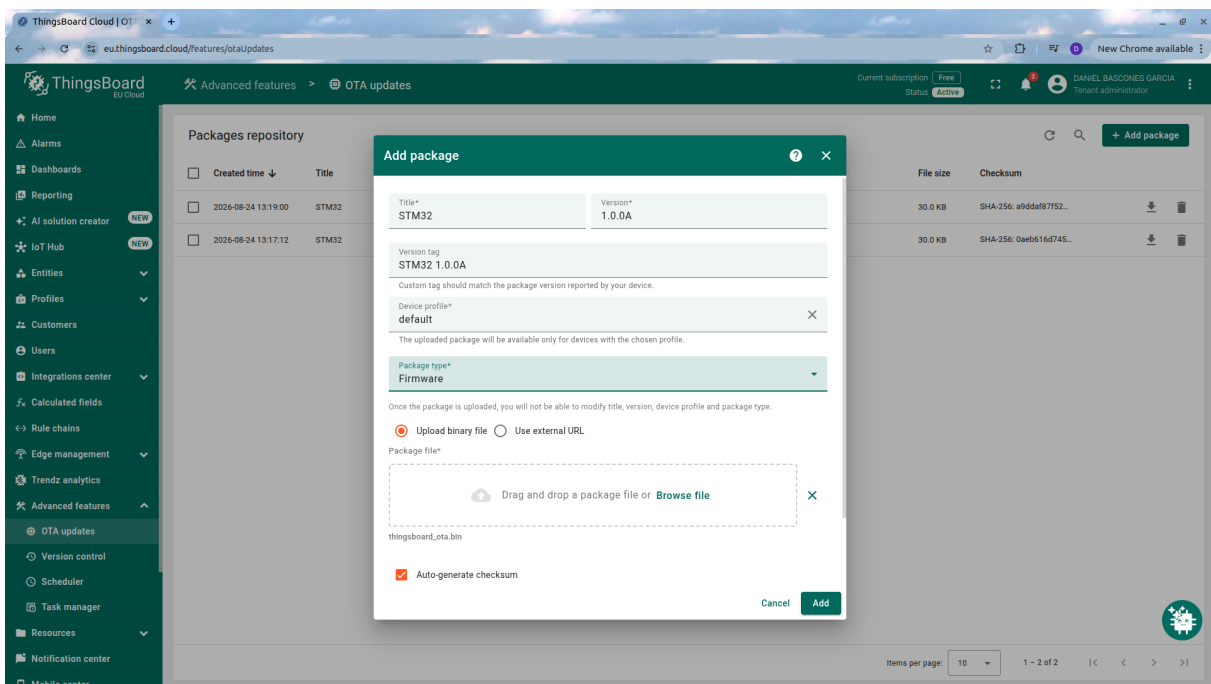


Figura 7: Subida de Firmware para OTA a *ThingsBoard*

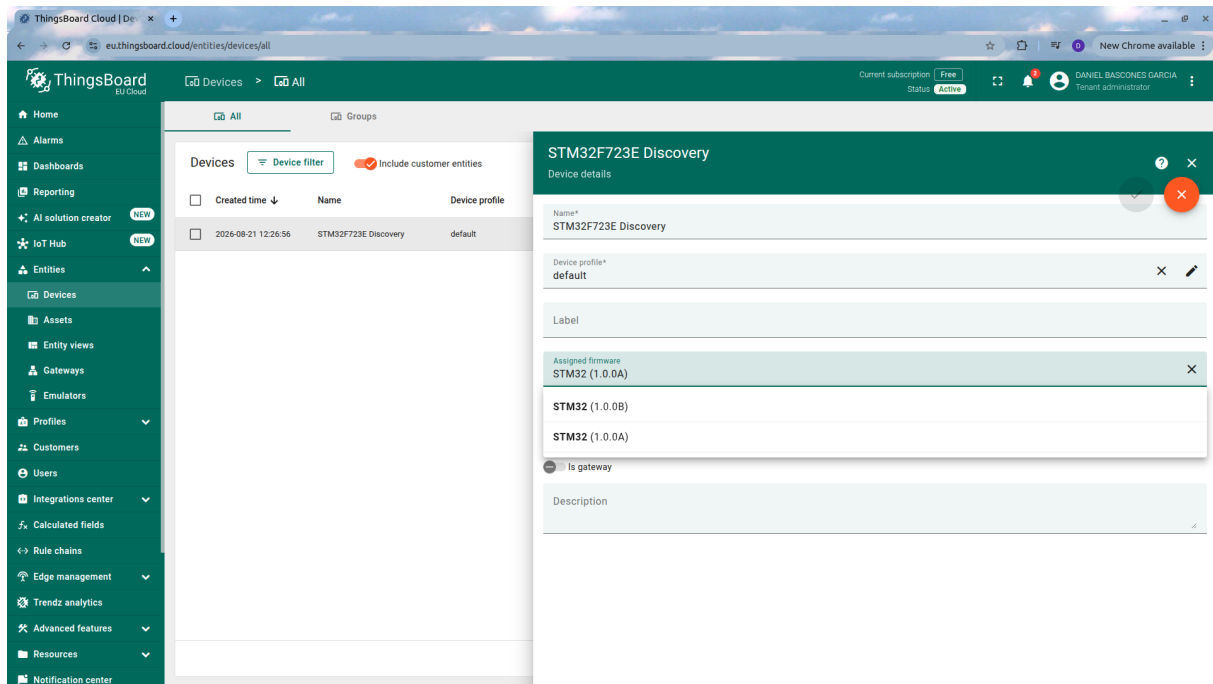


Figura 8: Subida de Firmware para OTA a *ThingsBoard*

```

4 [ESP] Setting Station Mode...
5 [ESP] Disabling MUX (Single Conn Mode)...
6 [ESP] Enabling Transparent Passthrough Mode...
7 [ESP] Connecting to SSID: guayfi...
8 [MQTT] Connecting to eu.thingsboard.cloud:1883...
9 [MQTT] Entering passthrough streaming...
10 [MQTT RX] CONNACK 0x00: Connection Accepted!
11 [MQTT RX] SUBACK Received! Granted QoS: 0
12 [MQTT] Connected & Subscribed successfully!
13 [SYSTEM] Ready! Streaming over MQTT...
14 [MQTT TX] {"temp":20.50,"counter":1,"touch_x":15,"touch_y":20}
15 [OTA] Rejected update 1.0.0A! Device is running in Slot A and requires B binary.

```

Podemos ver que, como el firmware seleccionado es para el mismo Slot, la aplicación lo rechaza y no actualiza nada. Si lo hiciera, colocaría un código que asume su inicio en `0x08000000` en la dirección de B `0x08040000`, donde cualquier salto absoluto resultaría en un fallo de ejecución. Si a continuación cambiamos el firmware objetivo desde *ThingsBoard* para nuestro dispositivo (Devices >Details >Assigned Firmware), la aplicación recibirá la actualización y, ahora sí, descargará la actualización:

```

1 [OTA] New Version Found: 1.0.0B (Size: 18628 bytes)
2 [OTA] Erasing Slot B (0x08040000)...
3 [OTA] Target Slot Erased!
4 [OTA RX] Chunk 1 (512 bytes) -> Total: 512 / 18628
5 // ..... Descarga de actualización
6 [OTA RX] Chunk 37 (196 bytes) -> Total: 18628 / 18628
7 // ..... Cambio de arranque
8 [OTA SUCCESS] Download Finished! Rebooting...
9 [OTA] Swapping BOOT_ADD0 to 0x0000000U (val: 0x08040000) and resetting...
10 // ..... Nuevo arranque desde SLOT B
11 [BOOT] Running App v1.0.0 from Address: 0x08040000
12 // ..... App en marcha! Ahora rechaza la aplicación nueva.
13 [OTA] Rejected update 1.0.0B! Device is running in Slot B and requires A binary.
14 [MQTT TX] {"temp":21.00,"counter":2,"touch_x":30,"touch_y":40}

```

Si volvemos a cambiar el Firmware por el A, la aplicación volverá a actualizarse gracias a los avisos que envía *ThingsBoard* con cada cambio de versión.

1.4. Para hacer más

En nuestro caso, estamos descargando “a ciegas” la aplicación, grabándola en la **FLASH**, y dándole el control con mucha confianza. Normalmente, al descargar una imagen binaria, también se envía un CRC o código de comprobación. Así, la aplicación puede calcular el CRC de los datos descargados, comprobarlo contra el CRC enviado por el servidor, y cambiar los bits de arranque únicamente si coinciden, asegurando la integridad de la app. Investiga cómo podría hacerse esto y qué cambios harían falta tanto en *ThingsBoard* como en nuestro código fuente.